

□ Binary Trees:

This presentation is about Binary Trees.

Presented by:

Syed Hassan 468

Ali Haider 474,

Hamza Razak 457



April 9, 2025.

☐ Topics Discuss:



What is a Binary Tree?



Types of Binary Trees.



Operations & Traversal.

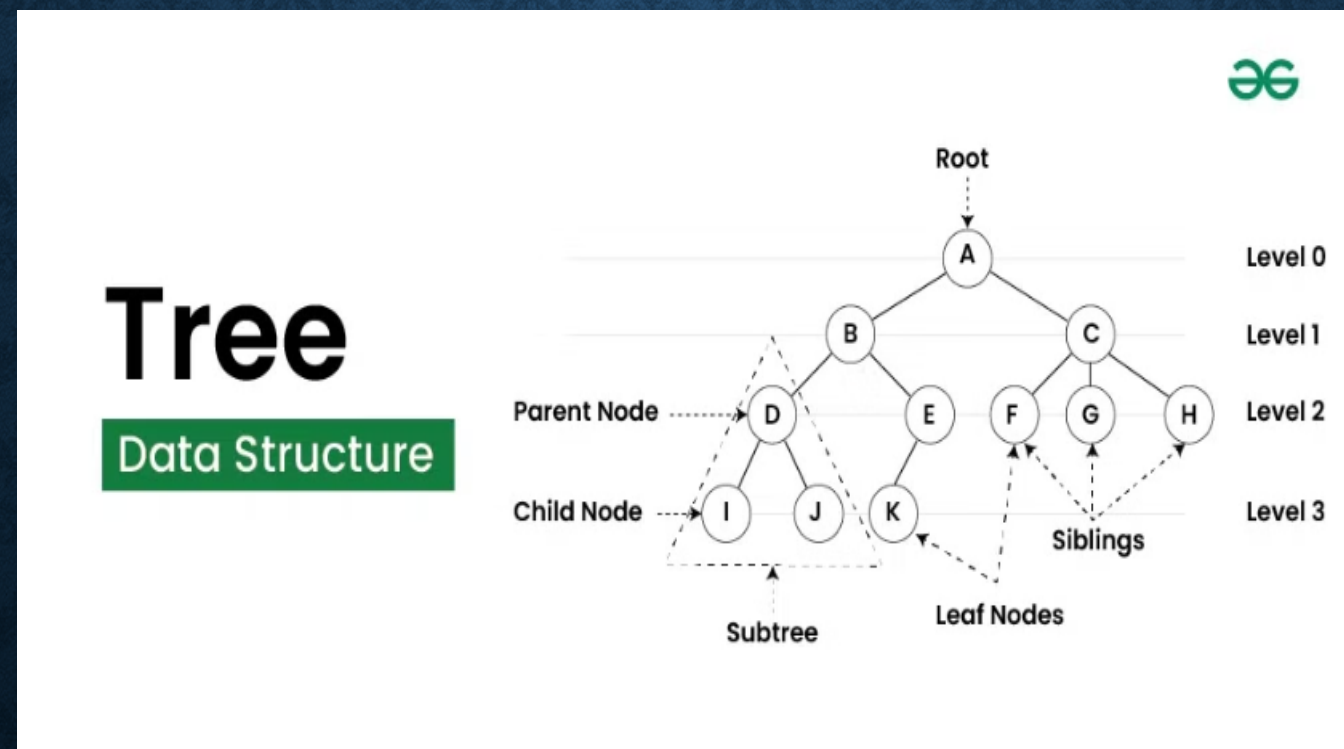


Applications & Properties.

□ What is a Tree?

Tree Structure

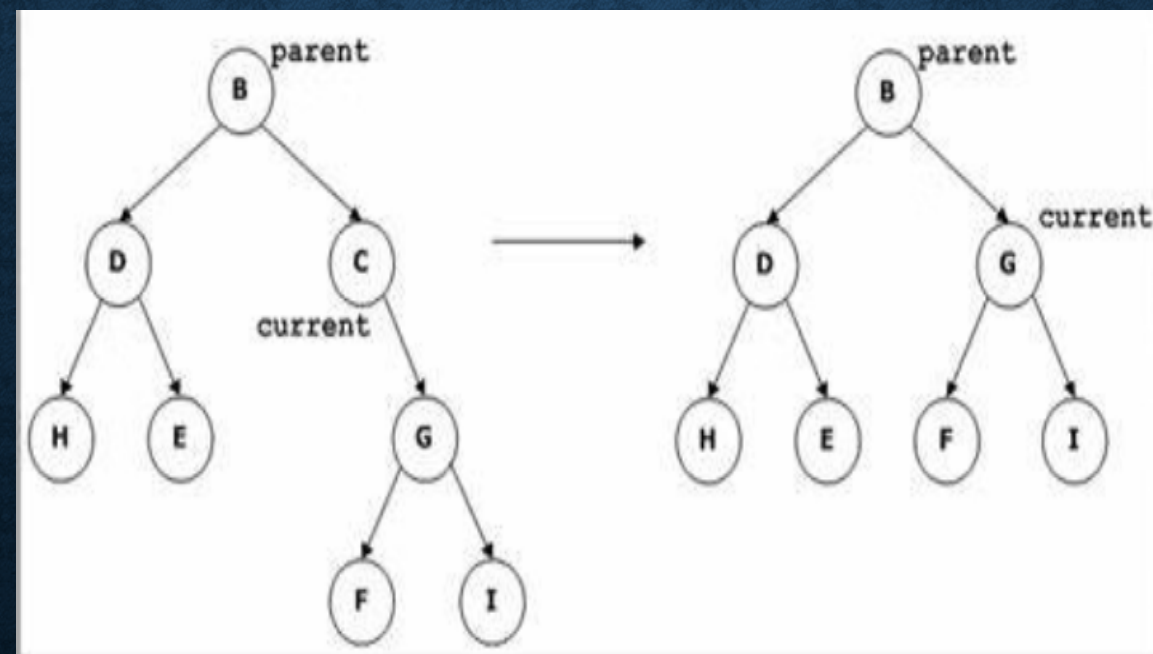
A tree is like a family tree or an organizational chart. It starts with one main node called the root. From the root, branches grow out to form other nodes, each representing a different part. Every node, except the root, has one parent node, creating a clear structure.



□ What is a Binary Tree?

Binary Tree Definition

A binary tree is a special type of tree where each node can have at most two children: a left child and a right child. This structure helps in keeping the tree balanced and organized. The left child typically holds smaller values, while the right child holds larger values. This property makes binary trees ideal for efficient searching and sorting and Fast searching, insertion, deletion. $O(\log n)$ when balanced.



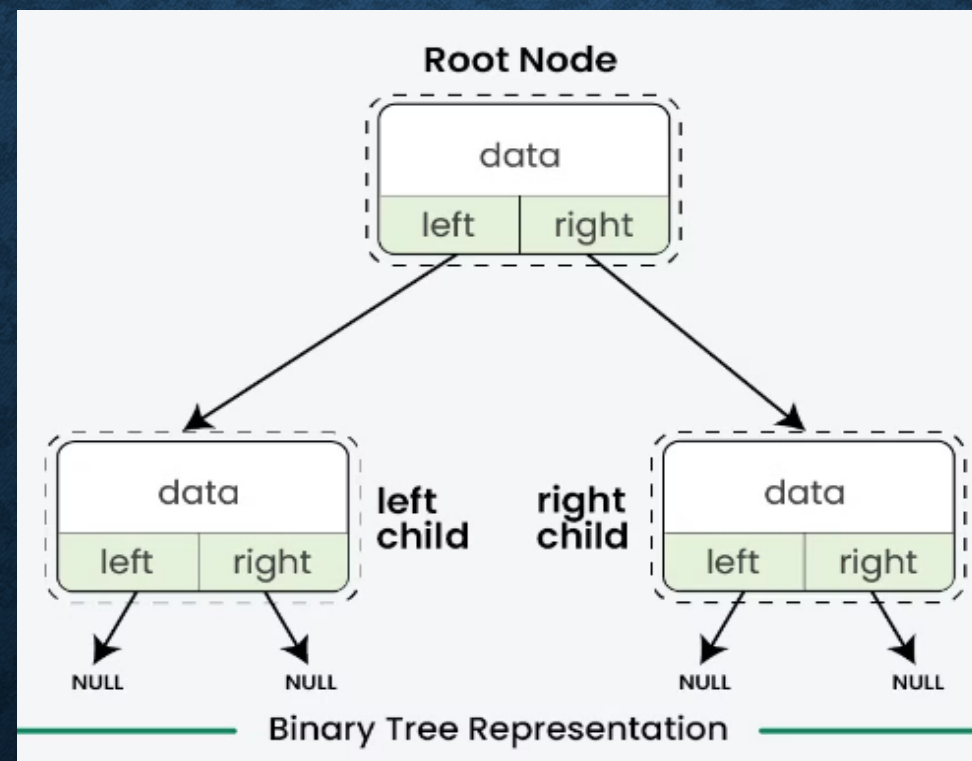
□ Understanding Binary Tree Structure:

Root: The top node with no parent.

Leaf: A node with no children.

Parent/Child: Connection between nodes.

Height: The longest path from root to leaf.



□Types of Binary Trees:

 Full Binary Trees

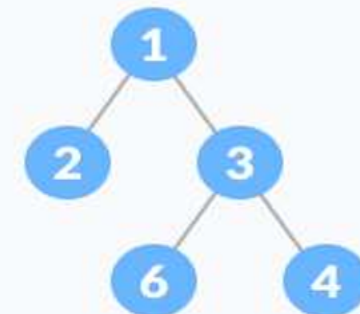
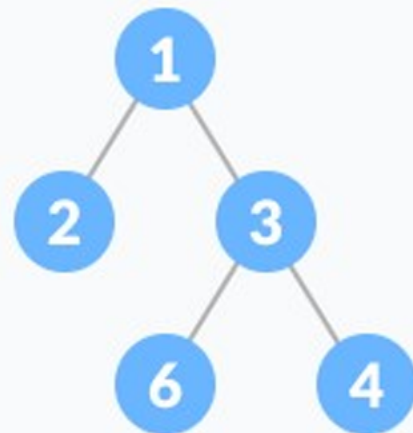
 Complete Binary Trees

 Perfect Binary Trees

□ Full Binary Tree

Full Tree

In a full binary tree, every node has either 0 or 2 children. This means no node has just one child. This balance helps keep the tree organized and efficient. It's especially useful in structures like heaps, where balance is important for quick operations. By having either no children or two children, the tree remains more stable.

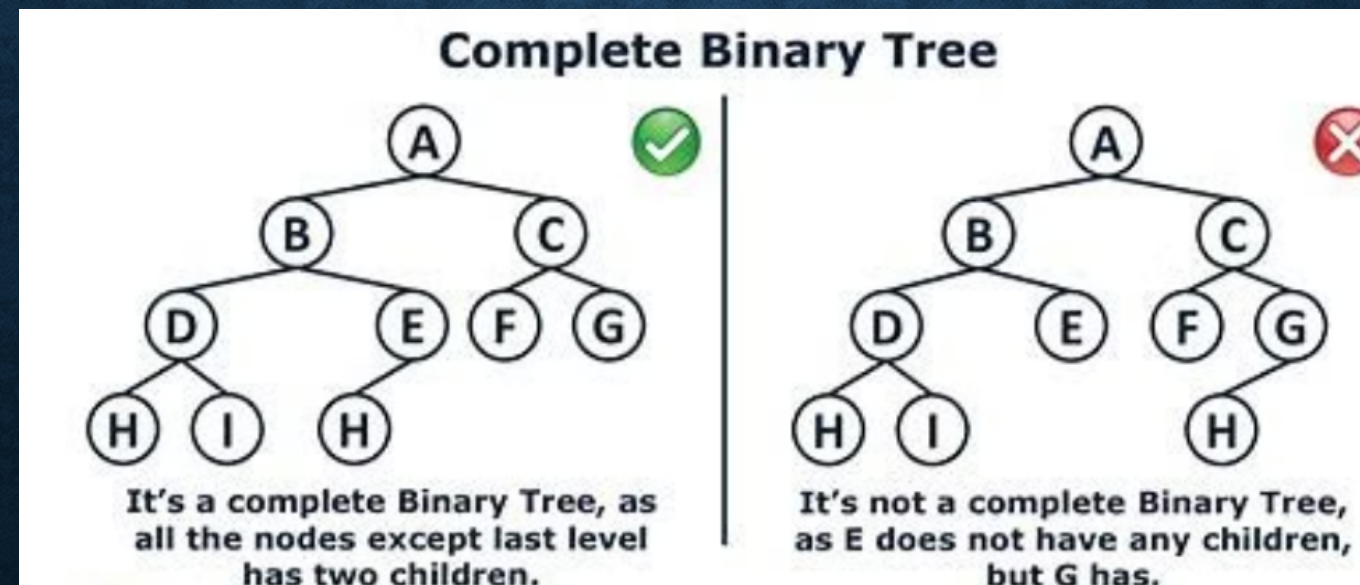


✓ Full Binary Tree
✗ Complete Binary Tree

□ Complete Binary Tree

Complete Tree

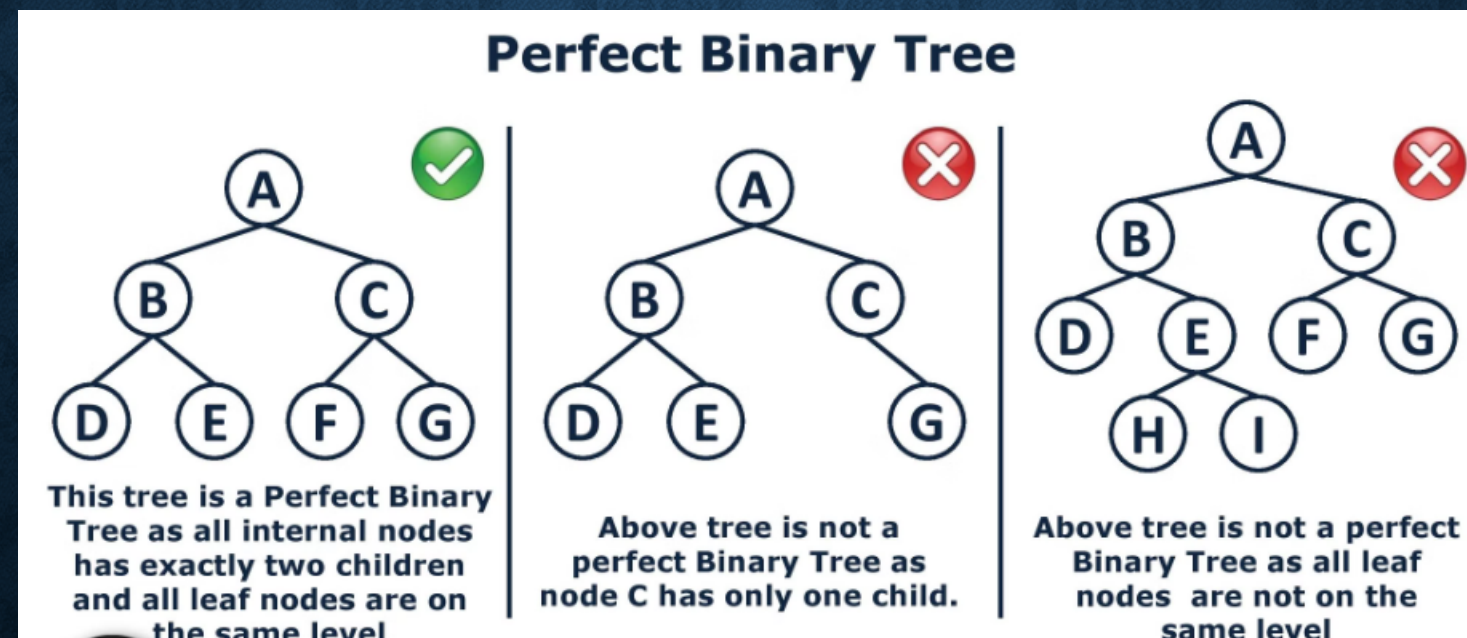
A complete binary tree is nearly full, with every level filled except possibly the last. The last level is filled from left to right. This structure helps in efficiently using memory and keeps things organized. It is especially useful in heaps, where fast access and memory usage are crucial. The complete binary tree ensures that data is stored in a balanced way, making operations faster.



□ Perfect Binary Tree

Perfect Tree

In a perfect binary tree, every internal node has exactly two children. All the leaf nodes are at the same level, making the tree perfectly balanced. This structure is highly efficient for search operations because it's evenly organized. However, to maintain this balance, the tree requires an exact number of nodes..



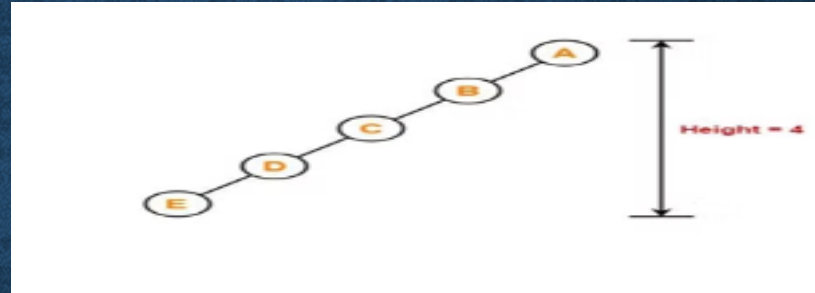
□ Height of Binary Search Trees (BST)

The height of a binary tree is the length of the longest path from the root node to any leaf node. It is measured in number of edges (not nodes).

□ Minimum Number of Nodes

- The **minimum** number of nodes in a binary tree of **height H** is:
- **$H + 1$**
- □ Example: For height **H = 4**, minimum nodes = **$4 + 1 = 5$**

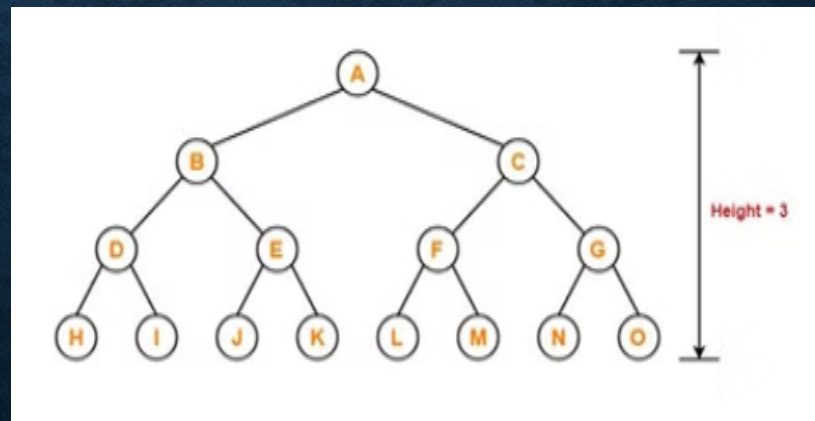
□ Example



□ Maximum Number of Nodes

- The **maximum** number of nodes in a binary tree of **height H** is:
- **$2^{(H+1)} - 1$**
- □ Example: For height **H = 3**:
- Max nodes = **$2^{(3+1)} - 1 = 16 - 1 = 15$ node**

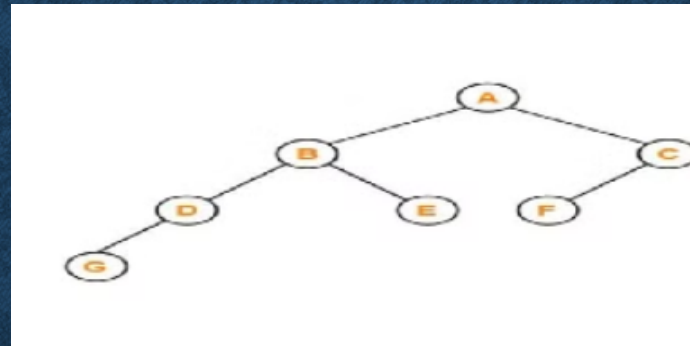
□ Example



□ Leaf Node Count

- Total **leaf nodes** = Total nodes with **2 children** + **1**
- □ Example:
- If a tree has **2 nodes** with 2 children →
- Leaf nodes = $2 + 1 = 3$

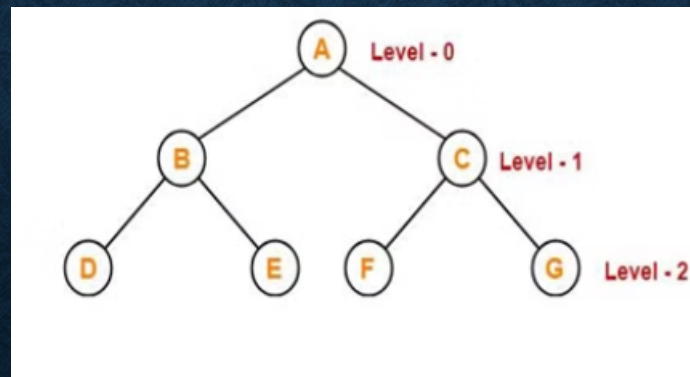
□ Example



□ Max Nodes at Level L

- Maximum number of nodes at **level L** = 2^L
- □ Example: At **level 2** → $2^2 = 4$ nodes

□ Example



□ Operations:

1) Insertion:

In a **Binary Search Tree (BST)**, smaller values go to the **left**, larger ones go to the **right**.
The structure must stay valid after inserting the new node.
It's usually done **recursively** or by **traversing** until the right spot is found.

2) Deletion:

In **Binary Search Tree (BST)** there are **three cases** to handle:

1. Node has **no children** – just delete it.
2. Node has **one child** – replace it with its child.
3. Node has **two children** – replace it with the **in-order successor** or **predecessor**.
4. After deletion, the tree needs to be **restructured** to stay valid.

3) Searching:

Searching means **finding a value** in the tree.
Start from the **root**, compare values, and move **left or right**.
In a balanced BST, searching takes **$O(\log n)$** time.
It's efficient because you **skip half the tree** at each step.

Tree Traversal:

- In trees, we **cannot access nodes randomly** like arrays or lists.
- So, we use special methods to visit all nodes in a specific order.
- There are **three main types** of traversal:

(a) Inorder (Left, Root, Right)

- Example: 4 2 5 1 3
- Used in **BSTs** to get data in **sorted order**.

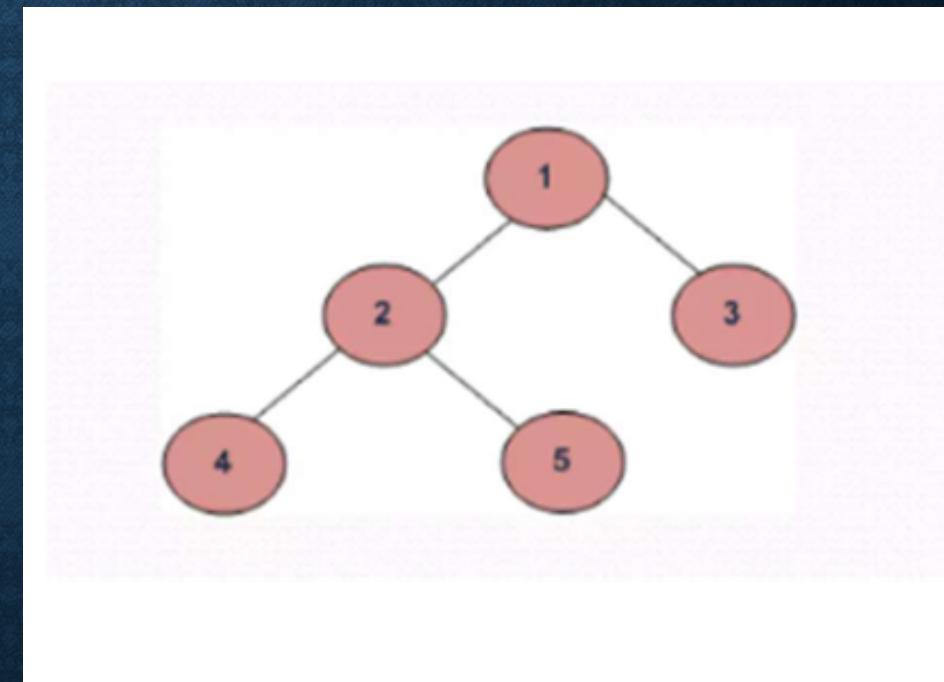
(b) Preorder (Root, Left, Right)

- Example: 1 2 4 5 3
- Useful for **copying a tree** or expression trees.

(c) Postorder (Left, Right, Root)

- Example: 4 5 2 3 1
- Often used in **deletion** and **evaluating expressions**.

Example



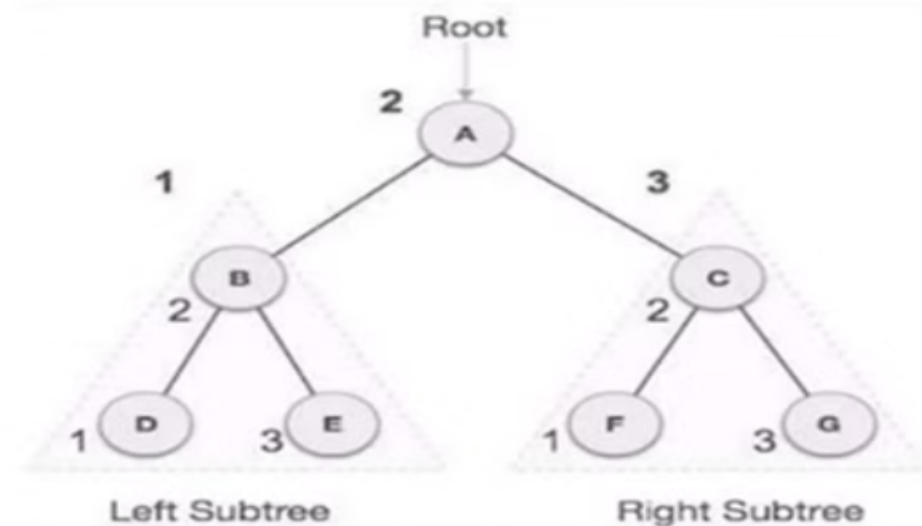
❑ Inorder Tree Traversal:

- In this traversal method, the **left subtree** is visited first,
- Then the **root**, and later the **right sub-tree**.
- We should always remember that **every node** may represent a **subtree** itself.

❑ Inorder Tree Traversal Algorithm:

Until all nodes are traversed → Recursively traverse the left subtree → Visit the root node. → Recursively Traverse the right subtree
Preorder Tree Traversal → In this traversal method, the root node is visited first, then the left subtree and finally right s

❑ Example



$D \rightarrow B \rightarrow E \rightarrow A \rightarrow F \rightarrow C \rightarrow G$

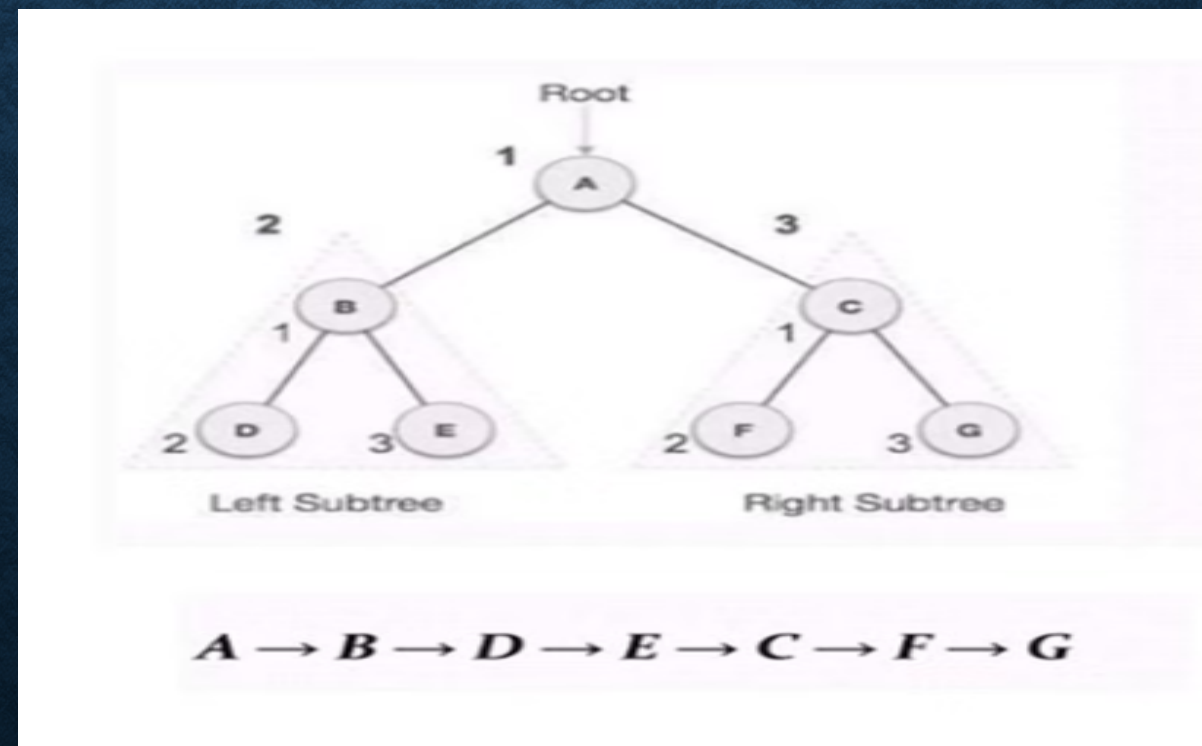
Preorder Tree Traversal

- In this traversal method, the **root node** is visited first,
- Then the **left subtree** and finally **right subtree**.

Preorder Tree Traversal Algorithm

- Until all nodes are traversed:
 - Visit the **root node**
 - Recursively **traverse the left subtree**
 - Recursively **traverse the right subtree**

Example



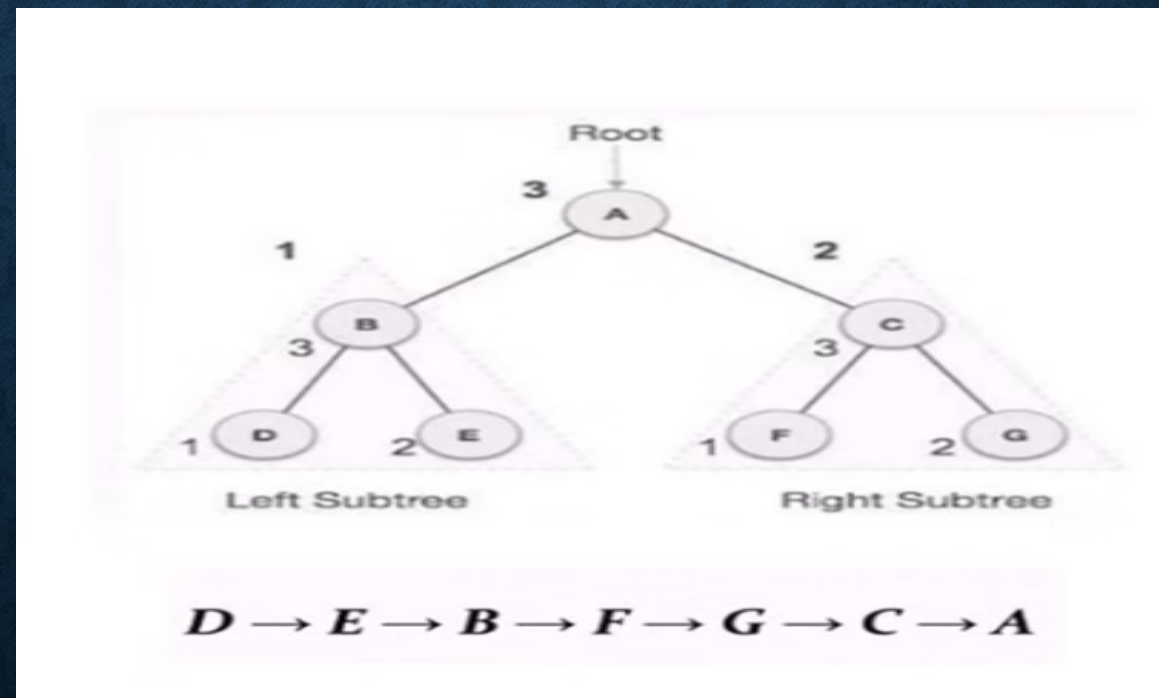
Post-order Tree Traversal

- In this traversal method, the **root node** is visited **last**.
- First we traverse the **left subtree**,
- Then the **right subtree**,
- And finally the **root node**.

Post-order Tree Traversal Algorithm

- Until all nodes are traversed:
 - Recursively **traverse the left subtree**
 - Recursively **traverse the right subtree**
 - **Visit the root node**

Example



Applications:

Feature / Operation	Binary Trees	Other Structures/Algorithms
Structure	Hierarchical, non-linear	Arrays: Linear, contiguous Linked Lists: Linear
Dynamic Insertions/Deletions	Efficient when balanced ($O(\log n)$)	Arrays: Costly due to shifting elements Linked Lists: $O(1)$ at ends
Search Efficiency	$O(\log n)$ when balanced (e.g., BST)	Hash Tables: Average $O(1)$ lookup, but unordered
Ordering	Maintains sorted order (via inorder traversal)	Arrays: Sorted if maintained Hash Tables: Unordered
Memory Usage	Non-contiguous allocation (nodes with pointers)	Arrays: Contiguous allocation Linked Lists: Extra pointers
Applications	Ideal for hierarchical data, decision-making, expression trees	Graphs: Model complex relationships Hash Tables: Fast key-value lookups

Thank You

Any Question?